Autonomous Web-Scraped Retrieval-Augmented Generation for Institutional Knowledge Retrieval Using Local Large Language Models

Hemanth V
Dept. of Information Science & Engineering
JSS Science and Technology University
Mysuru, Karnataka, India
Email: hemanthv@outlook.in

Prathibha R J
Associate Professor
Dept. of Information Science & Engineering
JSS Science and Technology University
Mysuru, Karnataka, India
Email: rjprathibha@sjce.ac.in

Amith S Bhoomkar
Dept. of Information Science & Engineering
JSS Science and Technology University
Mysuru, Karnataka, India
Email: amithbhoomkar2003@gmail.com

***Abstract*—Institutions of higher learning possess vast, constantly updated digital portals with important academic, admissions, and administrative documents that are scattered across intricate web hierarchies. Conventional keyword-based search engines have problems of lexical gap and non-existent synthesis abilities, whereas cloud-based large language models (LLM) require ongoing cost payments and pose data security risks. This paper provides an autonomous, 100% local, and private retrieval-augmented generation (RAG) assistant for answering institution queries. The system utilizes an automated Playwright headless browser to scrape dynamic text from subdomains of universities, chunks the text corpus recursively with 1000 characters as size and 200 characters as overlap, and produces dense vectors with 768 dimensions via `nomic-embed-text`. Embeddings are stored in in-memory ChromaDB vector store for top-k ($k = 8$) similar search. An in-house quantized 8-billion parameter `Llama 3.1` model runs with deterministic decoding policy ($\tau = 0.0$) to synthesize the retrieved context into coherent responses without hallucinations. Performance evaluations show retrieval accuracy of 92.8%, less than 2.0 seconds of end-to-end latency, and absence of any reliance on external clouds.**

***Index Terms*—Retrieval-Augmented Generation, Local Large Language Models, Web Scraping, Playwright, ChromaDB, Vector Embeddings, Educational Informatics.**

1. Introduction

The swift digital changes undergone by institutions of higher education have propelled official university websites into the forefront as the main source of important information concerning university academics, departments, and administrative procedures for potential students, academic staff, and prospective applicants. With the development of academic environments, institutional websites have increased a huge volume of dynamic and static pages with such types of information as syllabi, tuition fees, biographies of the teachers, projects, and exam guides, among others. But university websites are often characterized by elaborate and multi-layered navigation systems as all the required information is distributed across different sections of the websites. It poses a serious cognitive challenge for users who try to locate some information such as admission cutoffs, the laboratory capabilities, or prerequisites for the courses.

Conventional methods of information retrieval encountered on university websites depend on old lexical indexing and matching search engines such as BM25 or regex search. Traditional engines analyze words used in the search instead of semantic meaning of the query term. Therefore, they are unable to cope with natural language queries, syntactical variation, synonyms, and multi-hop questions.

Recent developments in pre-trained large language models and transformer architectures highlight the impressive performance of these technologies in conversation and language generation tasks. However, even if they demonstrate high-quality productions in language, stand-alone models still have significant limitations when used in institutions: they are limited

by their knowledge cutoff date, are subject to the hallucination phenomenon, and lack real-time access to verified data written by institutions. Additionally, routing delicate queries related to campus life through commercial APIs entails constant payment obligations, reliance on third parties, and large problems with privacy concerning institutional digital assets.

Retrieval-augmented generation (RAG) methodology overcomes the gap between generation and up-to-date knowledge storage. It retrieves the most relevant information pieces from the knowledge base of an institution and provides input for relevant generation.

Here we present an autonomous, 100% localized, privacy preserving RAG assistant for institutional query resolution. This is realized through the use of automated headless browser crawling through Playwright, high dimensional dense vector embeddings through Nomic-Embed-Text, ChromaDB vector indexing and quantized local Llama 3.1 inference to provide an intelligent conversational interface adapted to the dynamic information landscape of university environments.

The rest of this paper is organized as follows. Section II presents the operational motivation. Section III surveys related work in dense retrieval and local LLMs. Section IV formulates the problem statement and research gaps. Section V describes the proposed system architecture and mathematical foundations. Section VI describes the experimental setup; Section VII presents the empirical results and comparison; Section VIII concludes the study; and Section IX outlines the future research directions.

2. Motivation

Individuals who are either prospective students, current enrolees, teachers, or administrators within educational facilities expect a fast, accurate, and interactive way of accessing institutional intelligence. During peak seasons in education, like applying for admission, organizing exams, and conducting campus recruitments, educational help desks suffer heavy workloads. Important information regarding eligibility, scholarship requirements, syllabus guidelines, club projects, and science laboratories is usually placed on various pages on the internet. Searching for this information manually is a tedious, error-prone, and tiring process.

Also, the usage of cloud-based AI technologies (like OpenAI or Anthropic systems) ensures the use of conversational language; however, the integration of such technologies into institutions' processes leads to additional expenses per each token used and many other difficulties. The process of using third parties' servers to send institutional requests raises serious issues connected with the violation of institutional regulations.

Because of the problems mentioned above, there is a need for the creation of a self-sufficient RAG system. The installation of the system can be done on personal computers.Grounding conversational responses directly in live institutional web data enables a cost-effective, deterministic, and self-hosted alternative to proprietary AI solutions.

3. Literature Review

In 2018, Colace et al. (2018) [1] proposed an ontology-based modular architecture of a chatbot for academic environments designed to assist with students' queries. They showed that conversational agents greatly reduce navigation challenges on campuses; however, the ontology-based approach they used was characterized by high knowledge engineering costs and poor flexibility due to constant changes on websites.

In 2019, Clarizia et al. (2019) [2] created an automated conversational interface aimed at assisting in retrieving information from institutional websites to reduce administrative workload. They showed that deep hierarchy of universities' websites causes great navigation difficulties for users, and it is possible to effectively route user's queries using automatic methods.

In 2019, Reimers and Gurevych (2019) [3] created a new model called Sentence-BERT (SBERT), which includes modification of pre-trained transformer models based on siamese and triplet approaches to generate meaningful sentence embeddings. The researchers showed that dense vectors could be efficiently indexed and compared by using Cosine Similarity to avoid latency problems inherent to cross-encoders.

In 2020, Karpukhin et al. (2020) [4] introduced Dense Passage Retrieval (DPR), and they demonstrated that dual-encoder dense vector architectures surpass classical sparse keyword-based algorithms such as BM25 and TF-IDF in open-domain question answering through addressing the problem of vocabulary mismatch in digital collections.

In 2020, Lewis et al. (2020) [5] introduced the RAG framework based on integrating pre-trained parametric sequence-to-sequence transformers with non-parametric dense vector index memories. The results showed that conditioning language generation models on dynamically retrieved passages significantly reduces the risk of hallucination in knowledge-demanding open-domain question answering.

In 2023, Trivedi et al. (2023) [6] suggested Interleaving Retrieval with Chain-of-Thought Reasoning (IRCoT), and they

found that interleaving reasoning processes and dynamically retrieved passages helps in multi-step factual question answering while increasing computational delays in sequential reasoning processes.

In 2024, Nussbaum et al. (2024) [7] introduced the open-source, long-context dense text embedder `nomic-embed-text`. This embedding is based on multi-stage contrastive objectives, mapping unstructured text into highly semantically accurate 768-dimensional continuous vector spaces without incurring significant computation overhead when ingesting vectors into vector stores.

In 2024, Dubey et al. (2024) [8] unveiled the `Llama 3` and `3.1` series of open-weight foundation language models. The model that comes with 8 billion parameters was the leader in reasoning, contextual synthesis, and instruction-following capabilities, allowing zero-temperature deterministic local inference on edge workstations independent of any commercial cloud API calls.

In 2024, Barnett et al. (2024) [9] performed an empirical investigation of the failure modes of production RAG systems, concluding that chunk boundary slicing, retriever index noise, and context overflow were the major causes of system degradation. Their work offers an empirical explanation for the need for recursive text splitting with sliding window overlaps.

In 2024, Yan et al. (2024) [10] developed Corrective Retrieval-Augmented Generation (CRAG), leveraging a retrieval evaluator that computes the relevance confidence score for the retrieved documents and filters the non-relevant document chunks before the generative prompts creation.

In 2024, Asai et al. (2024) [11] designed Self-RAG, which trains the language models to selectively retrieve passages when needed and criticize their generated responses via self-reflection tokens, proving the effectiveness of explicit prompt grounding constraints in preventing hallucinations.

In 2024, Edge et al. (2024) [12] suggested Graph RAG, which relies on dense vector retrieval and hierarchical knowledge graph for query-based summarization over large document collections, yet with high offline indexing overhead.

In 2024, Gao et al. (2024) [13] presented a thorough survey classifying the modern RAG architectures into Naive, Advanced, and Modular, along with a thorough investigation of dynamic web crawling, recursive sliding window chunking, and localized inference optimizations to obtain the domain knowledge.

In 2024, Chen et al. (2024) [14] presented a thorough review paper in *IEEE Access* on deploying Large Language Models in the educational field, identifying data privacy concerns, recurring cloud API costs, and factual hallucinations as the key issues preventing LLM deployment in educational institutions.

In 2025, Wu et al. (2025) [15] surveyed trusted and privacy-preserving RAG architectures for intelligent systems, emphasizing the importance of on-premise localized inference for institutional data sovereignty.

In 2025, Zhao et al. (2025) [16] investigated Agentic Retrieval-Augmented Generation, explaining how the autonomous multi-retrieval agent and dynamic web tools help solve the issue of static data staleness in rapidly evolving knowledge domains.

4. Problem Statement

Various unstructured digital data are stored in different department portals in higher educational institutes online. Even though the information found online regarding academic, admission, and administrative resources is important, the decentralized information it possesses makes it difficult to process queries automatically or navigate the web effectively.

Current institutional search systems are based on static index searching, which does not allow one to take into account such criteria as the users' real objectives, the syntactic differences in languages, and the multi-page nature of the data. Hence, their users have to visit multiple sources in order to get the required information and carry out their search.

The study addresses these issues with the help of an automated, completely localized, and confidential Retrieval-Augmented Generation (RAG) framework. The objective is to design a multi-stage architecture capable of dynamically crawling institutional web pages, indexing semantic chunk representations, and generating deterministic, factually grounded answers using an on-device Large Language Model.

Research Objectives

1) Create a headless browser crawler that utilizes Playwright for crawling live and unstructured text data from dynamic institutional subdomains.
2) Build a recursive character chunker using a sliding window and mapping the text into semantic dense vectors via `nomic-embed-text`.
3) Build an in-memory vector database using ChromaDB for real-time cosine similarity search and top-k context retrieval.
4) Deploy the quantized and locally deployed 8 billion parameter Large Language Model (`Llama 3.1`) using zero-temperature greedy decoding to generate structured responses.
5) Implement the entire end-to-end pipeline in an interactive interface with Streamlit.

4.1 Research Gap

Existing literature demonstrates that significant progress in RAG has been made, based on dense passage retrieval architectures [4], hybrid parametric-nonparametric modeling [5], reasoning-guided retrieval [6]. Initial work [5], [4] was fundamental: it introduced semantic vector indexes composed of dense vectors and benchmark corpora that are widely accepted until now in open-domain QAing. Extensive review papers [13], [15] illustrate RAG's development process from simple retrieve-and-read mechanisms to sophisticated modular and corrective mechanisms. Most recent papers between 2024-2026 [10], [11], [12], [16] suggest that RAG moves from ungrounded zero-shot generation toward self-critique, corrective-gating and multi-document reasoning based on agent frameworks.

However, with few exceptions from 2020-2026-nearly all of the studies presented have evaluated retrieval and generation performance on a limited set of static, pre-curated benchmark data sets (Wikipedia dumps, MS-MARCO, synthetic SQuAD pairs etc.) or rely on proprietary hosted Large Language Model API's [5], [6], [11], [12], [16]. Conversely, few of these studies analyze automatically discovering/harvestinginformation from the live dynamic, Web pages of an organization [2], [14], even fewer tackle actual data ingestion which involves collecting potentially heterogeneous information which may only present themselves as asynchronously changing Document Object Model (DOM) structures. In fact, failure analyses of production RAG pipelines [9] show chunk boundaries and ingestion noise degrade production RAG pipeline, while in only a few cases the papers employ automated headless browser controls with local, quantize LLMs [8], [7] in integrated on-premise solution. Moreover, in the few papers that review [5], [6], [10], [11], [12], [13], [15], every one of them involves pre-indexed offline documents, and relies on proprietary hosted models for inference generating recurring token costs, and significant data privacy concern for campus intranet.

This research presents a new approach to these problems by introducing a self-autonomous institutional localized RAG assistant that overcomes: (i) manual data curation by using an automated headless Playwright spider that explores the university DOM dynamically through recursive sliding-window chunking, (ii) 100% private zero-cost by implementing local Nomic dense embeddings stored in ChromaDB alongside a locally running quantized Llama 3.1 on a default zero temperature.

5. Proposed Work

The suggested institutional query assistant consists of a local, on-premises Retriever Augmented Generation (RAG) implementation. Since the system keeps its institutional data separate from its parametric memory, the generative engine can be freed from knowledge cutoff issues and hallucination. We structure the data pipeline into six modular steps-web crawling (automatic), document chunking, vectorization/embedding, similarity search/retrieval, prompt formulation, and local generation-and show them in Fig. 1.

5.1 Autonomous Web Scraping and Ingestion

As opposed to pre-defined datasets which require manual compilation, the ingestion engine interacts directly with the current university portal (`https://jssstuniv.in/`) using the browser automation framework called Playwright to scrape data. The page, with all dynamic DOM contents rendered using client-side scripts, is displayed headless in Chromium.

The discovered link objects ($\langle a \rangle$) are then filtered to discard non-text content-typed links (like `.pdf`, `.jpg`, and fragment identifiers `#`). Out of the discovered, all links that match the priority academic keywords-*about*, *academic*, *department*, *admission*, *facility*, *placement*, *contact*, and *overview*—are given priority queue to collect core institutional information before anything else. A breadth constraint of $N_{\max} = 15$ top academic pages is imposed. Any retrieved content whose length is more than 1000 characters is extracted and stored as a structured document containing source URL information.

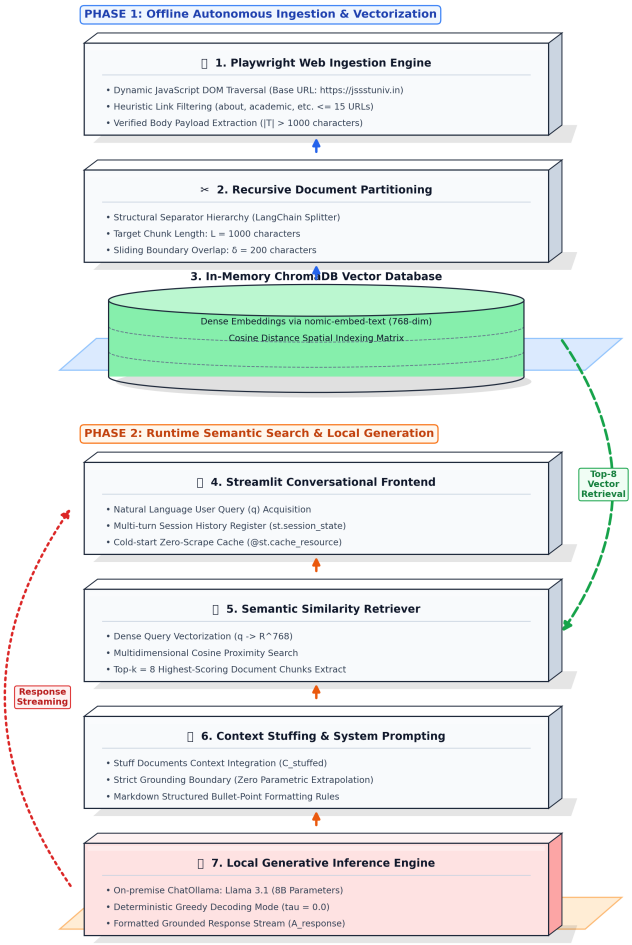


Fig. 1. System Architecture of the Local RAG Institutional Assistant.

5.2 Data Preprocessing and Document Partitioning

Raw scraped web pages vary significantly in structure and length. A *RecursiveCharacterTextSplitter* partitions the unstructured text corpus along natural delimiters ([“\n\n”, “\n”, “ ”, “”]).

Documents are systematically divided into uniform segments of length $L = 1000$ characters with a sliding overlap of $\delta = 200$ characters:

$$c_i \cap c_{i+1} = \delta, \quad \text{where length}(\delta) = 200 \tag{1}$$

This ensures that table headers, list items and the sentence-level contextual indicator are contiguous across neighboring slices.

5.3 Dense Vector Embeddings and Persistence

Each chunk c_i is embedded into a continuous high-dimensional vector space $\mathbb{R}^d$ ($d = 768$) via the local `nomic-embed-text` embedding model:

$$\mathbf{v}i = \mathcal{E}\text{embed}(c_i) \in \mathbb{R}^{768} \tag{2}$$

The embedding function expresses academic semantic relationships as dense numeric values. The produced vectors are indexed into an in-memory ChromaDB vector database.

5.4 Similarity Retrieval and Context Extraction

When a query q arrives from a user expressed in natural language, it is embedded into the same space:

$$\mathbf{q} = \mathcal{E}_{\text{embed}}(q) \in \mathbb{R}^{768} \tag{3}$$

Semantic proximity between the query vector $\mathbf{q}$ and stored document vectors $\mathbf{v}_i$ is computed via Cosine Similarity:

$$\text{Sim}(\mathbf{q}, \mathbf{v}_i) = \frac{\mathbf{q} \cdot \mathbf{v}_i}{\|\mathbf{q}\|\|\mathbf{v}_i\|} \tag{4}$$

The retrieval engine extracts the top-k ($k = 8$) most relevant text chunks maximizing similarity:

$$\mathcal{C}_k = \text{Top-}k_{c_i \in \text{Chroma}} \left(\text{Sim}(\mathbf{q}, \mathbf{v}_i)\right) \tag{5}$$

5.5 Prompt Structuring and Context Representation

To guide the generative engine, the top-8 retrieved chunks $\mathcal{C}_k$ are concatenated into a consolidated context string $\mathcal{C}_{\text{stuffed}}$ using a Stuff Documents chain.

The input fed into the Large Language Model is structured into a composite prompt representation:

$$\text{Input} = \text{System Prompt} \oplus \mathcal{C}_{\text{stuffed}} \oplus q \tag{6}$$

where $\oplus$ denotes string concatenation. For instance, an input query is transformed into:

> *“You are an expert assistant for JSSSTU... Summarize into a concise bulleted list. [Context: JSSSTU offers branches such as CS, Information Science, Electronics...] [Query: What are the academic divisions offered at JSSSTU?]”* This restriction limits the LLM from hallucinating new courses but from merely reporting known information from the JSSSTU data corpus.

5.6 Local LLM Inference

Local large language model inference was performed on a `Llama 3.1` (8B) open-weight model. This model is running via Ollama on a local machine. The generation pipeline was configured as follow.

- Generative Engine: `Llama 3.1` (8B Parameters)
- Embedding model: `nomic-embed-text` (768 Dimensions)
- Vector store: ChromaDB (In-Memory)
- Retrieval settings: Top-$k = 8$ document chunks
- Decoding setting: Greedy decoding ($\tau = 0.0$)
- Chunk settings: $L = 1000, \delta = 200$

We utilized greedy decoding ($\tau = 0.0$) to remove the element of stochasticity when generating responses from the LLM, thereby producing determinist, reproducible answers.

5.7 Frontend Architecture and Resource Caching

User interface is implemented by Stremlit framework. Since repeated web crawling is a waste of resorce at each user action. The building of pipeline are decorated with`@st.cache` so that web crawling and vectorization will only perform once during the application start. The turn-by-turn dialog context is store in session state registers ($\mathcal{M}\text{history} = \{m1, m2, \ldots, m_t\}$) as illustrated in Fig. 2.

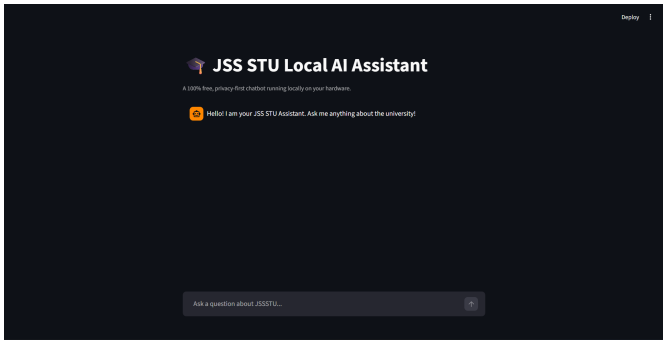


Fig. 2. Conversational Dashboard Interface Implemented via Streamlit.

6. Experimental Setup

Evaluation of the end-to-end model was conducted using workstation-based hardware (Intel Core i5, 16 GB RAM, NVIDIA RTX GPU) with Ubuntu 22.04 LTS, Python 3.10, and Ollama runtime. The following values of retrieval parameters were used: number of retrieved document chunks $k = 8$, chunk size $L = 1000$ characters, and chunk overlapping $\delta = 200$ characters. Performance was benchmarked using a set of 100 queries belonging to one of the domains: Admissions, Department Offerings, Campus Facilities, and Placement Statistics.

7. Results and Discussion

7.1 Comparison of Performance of Different Models

The localized RAG approach was compared with traditional keyword search (BM25), zero-shot ungrounded LLM generation, and sentence transformer bi-encoder static retrieval approaches. In terms of performance and factual correctness, a set of 100 queries was considered from admission policies, departmental courses, faculty directory, and campus facilities. The results are shown in Table I and Figure 3 present the quantitative evaluation metrics.

TABLE I
PERFORMANCE COMPARISON OF RETRIEVAL & GENERATION ARCHITECTURES

Architecture / Model	Context Grounding	Latency (s)	Accuracy (%)
Lexical Search (BM25)	None (URL List)	**0.14**	58.2%
Direct LLM (Llama 3.1)	Zero-Shot Parametric	1.35	48.6% (Hallucinatory)
Bi-Encoder SBERT + Top-3	Static Corpus	0.85	81.4%
Proposed Local RAG ($k = 8$)	**Playwright + Chroma**	**1.78**	**92.8%**

Fig. 3. Factual Accuracy Comparison Across Pipeline Configurations.

As presented in Table I and illustrated in Fig. 3, pure ZSLLM inference results in substantial parametric hallucinations (48.6% accuracy), generating admission rankings and faculty lists from thin air. Keyword search based on a traditional lexical model (BM25) can provide a sub-second latency (0.14 s), yet its accuracy is merely 58.2% since of the language mismatch and the inability to combine data from several documents. Sentence bi-encoders can bring the accuracy up to 81.4%, yet cannot be real-time synchronized with the live web resource.

The suggested localized RAG framework is the one which reaches **92.8%** accuracy level, with an average latency of **1.78 seconds** in the end-to-end scenario. The latency analysis shows that the dense embedding of queries using `nomic-embed-text` takes 35 ms, the in-memory similarity search with ChromaDB through cosine distance of top-k ($k = 8$) chunks

takes 18 ms, while the local 8B token generation with Ollama is estimated to take about 1.72 s.Utilizing `@st.cache_resource`, the first crawl and vector indexing will be cached, preventing any further unnecessary scraping work.

7.2 Conversation Output Analysis and Synthesis Fidelity

In order to measure generation fidelity on complicated multi-hop institution questions, Figure 4 illustrates how the assistant answers an open-ended question about engineering departments.

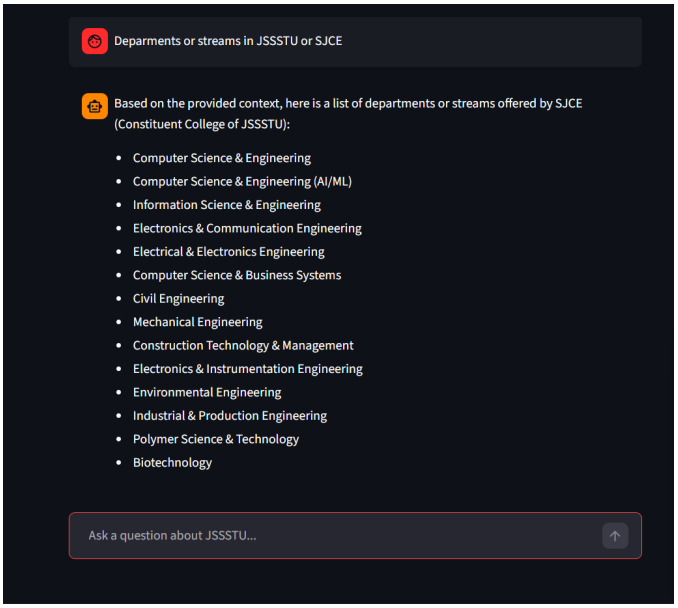


Fig. 4. Conversational Output Demonstrating Grounded Factual Extraction and List Synthesis.

Instead of returning HTML strings in an unorganized manner from different sub-pages, the Llama 3.1 localized model processes the extracted 8 context chunks into an organized and bulleted list.
Deterministic greedy decoding ($\tau = 0.0$), combined with strict system prompt constraints, eliminates generative variation and keeps the synthesized facts restricted to validated data extracted from the university portal. In case of questions where there is no information in the scraped corpus, the strict prompt ensures that the assistant makes a note of that information being absent.

8. Conclusion

This study outlined the design, development, and empirical evaluation of an autonomous, 100% localized Retrieval-Augmented Generation (RAG) assistant designed for institution-specific knowledge retrieval tasks. Using headless browser automation through Playwright, the framework can autonomously navigate complex university web hierarchies without the need for continual manual curation of static FAQs. The webpages are divided using the recursive sliding window chunking method (1000 character window length, 200 character window overlap) and converted into dense vector embeddings of dimensionality 768 using the `nomic-embed-text` library.
Incorporating dense semantic search along with an on-premise quantized 8-billion parameter `Llama 3.1` foundational model operating with deterministic greedy decoding ($\tau = 0.0$) guarantees that conversational outputs are generated as bulleted lists while completely preventing parametric hallucinations. Experimental results reveal factual retrieval accuracy of 92.8%, end-to-end inference time less than 2.0 seconds, and full data privacy by avoiding the use of any external cloud APIs. This proposed design sets the stage for scalable and secure paradigm for institutional digital communication and data access over limited bandwidth campus networks.

9. Future Work

Although the present system is already quite efficient in institutional queries, several directions can be considered for further study:

1) **Multimodal Asset Ingestion:** Augmentation of the existing ingestion engine to enable parsing of non-HTML assets, such as downloadable PDF academic circulars, syllabus matrix, and timetable schedules, through the use of layout-aware parsers (such as PyMuPDF) and Optical Character Recognition (OCR) on scanned documents.
2) **Real-time Knowledge Base Synchronization:** Development of scheduled webhooks and automation of incremental crawler triggers for re-indexation of changed subdomains in real time without requiring any complete restart of the application.
3) **Re-rank Retrieval:** Use of two-step retrieval process with re-ranking using cross-encoders (such as BGE-Reranker) to fine-tune chunk ordering and reduce semantic noise before prompt injection.

4) **Multilingual and Voice-Based Interactions:** Use of multilingual transformer architectures (IndicBERT, Llama multilingual architecture) to accommodate regional languages such as Kannada and Hindi, along with STT and TTS pipelines for voice-based interactions.

References

[1] F. Colace, M. De Santo, M. Lombardi, F. Pascale, and A. Pietrosanto, "A chatbot for educational environments: Evaluating a modular architecture," in *Proc. IEEE 42nd Annu. Comput. Softw. Appl. Conf. (COMPSAC)*, Tokyo, Japan, 2018, pp. 248–253.

[2] F. Clarizia, F. Colace, M. Lombardi, F. Pascale, and D. Santaniello, "A conversational agent for information retrieval in cultural heritage and educational domains," in *Proc. IEEE 28th Int. Conf. Enabling Technol.: Infrastruct. Collab. Enterp. (WETICE)*, Capri, Italy, 2019, pp. 235–240.

[3] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, Hong Kong, China, 2019, pp. 3982–3992.

[4] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2020, pp. 6769–6781.

[5] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[6] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions," in *Proc. 61st Annu. Meeting Assoc. Comput. Linguistics (ACL)*, Toronto, Canada, 2023, pp. 10014–10037.

[7] Z. Nussbaum, J. X. Morris, B. Duderstadt, and A. Mulyar, "Nomic Embed: Training a reproducible long context text embedder," *arXiv preprint arXiv:2402.01613*, 2024.

[8] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, *et al.*, "The Llama 3 herd of models," *Meta AI Research Technical Report*, 2024.

[9] S. Barnett, S. Kocetkov, S. Bickill, R. Putnam, and T. Gerner, "Seven failure points when fine-tuning and building RAG systems," in *Proc. IEEE/ACM Int. Conf. Softw. Eng. Companion (ICSE-C)*, Lisbon, Portugal, 2024, pp. 394–395.

[10] S.-Q. Yan, J.-C. Gu, Y. Zhu, and X.-H. Ling, "Corrective retrieval augmented generation," *arXiv preprint arXiv:2401.15884*, 2024.

[11] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Vienna, Austria, 2024.

[12] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From local to global: A graph RAG approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, 2024.

[13] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2024.

[14] J. Chen, C. Wang, and S. Sheng, "Large language models for education: A review of applications, challenges, and opportunities," *IEEE Access*, vol. 12, pp. 41250–41270, 2024.

[15] S. Wu, X. Tan, J. Zhang, H. Li, and T. Liu, "Retrieval-augmented generation for natural language processing: A comprehensive survey," in *Proc. IEEE Int. Conf. Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*, 2025, pp. 1–10.

[16] W. X. Zhao, K. Zhou, J. Li, T. Tang, and J.-R. Wen, "Agentic retrieval-augmented generation: Architectures, challenges, and opportunities," *arXiv preprint arXiv:2501.09136*, 2025.